Node.js Lane (Express with Swagger UI)

To get Swagger UI running in Express, you need to install the package and provide an OpenAPI specification file (`openapi.json`).

1. Install the package

Run this command in your terminal:

bash

```
npm install swagger-ui-express
```

Use code with caution.

2. Create the specification file

Create a file named `openapi.json` in your project root directory and paste this configuration:

json

```
{
  "openapi": "3.0.0",
  "info": {
    "title": "Task API",
    "version": "1.0.0",
    "description": "A simple CRUD Task Management API"
  },
  "paths": {
    "/": {
      "get": {
        "summary": "Get API metadata",
        "responses": {
          "200": { "description": "Returns API name and version info" }
        }
      }
    },
    "/health": {
      "get": {
        "summary": "Check API health status",
        "responses": {
```

```
        "200": { "description": "Returns healthy status confirmation" }
      }
    }
  },
  "/tasks": {
    "get": {
      "summary": "List all tasks",
      "responses": {
        "200": { "description": "A list of tasks" }
      }
    },
    "post": {
      "summary": "Create a new task",
      "requestBody": {
        "required": true,
        "content": {
          "application/json": {
            "schema": {
              "type": "object",
              "required": ["title"],
              "properties": {
                "title": { "type": "string" }
              }
            }
          }
        }
      },
      "responses": {
        "201": { "description": "Task created successfully" },
        "400": { "description": "Invalid input provided" }
      }
    }
  },
  "/tasks/{id}": {
    "get": {
      "summary": "Get a task by ID",
      "parameters": [
        { "name": "id", "in": "path", "required": true, "schema": { "type": "integer" }
      ],
      "responses": {
        "200": { "description": "Task found" },
        "404": { "description": "Task not found" }
      }
    },
```

```
      "put": {
        "summary": "Update an existing task",
        "parameters": [
          { "name": "id", "in": "path", "required": true, "schema": { "type": "integer" }
        ],
        "requestBody": {
          "content": {
            "application/json": {
              "schema": {
                "type": "object",
                "properties": {
                  "title": { "type": "string" },
                  "done": { "type": "boolean" }
                }
              }
            }
          }
        },
        "responses": {
          "200": { "description": "Task updated successfully" },
          "400": { "description": "Invalid body update payload" },
          "404": { "description": "Task not found" }
        }
      },
      "delete": {
        "summary": "Delete a task by ID",
        "parameters": [
          { "name": "id", "in": "path", "required": true, "schema": { "type": "integer" }
        ],
        "responses": {
          "204": { "description": "Task deleted successfully" },
          "404": { "description": "Task not found" }
        }
      }
    }
  }
}
```

Use code with caution.

3. Wire Swagger UI into your code

Open `server.js`. Import your `openapi.json` file and setup the `/docs` route using the package.

Add these lines near the top of your `server.js` file (right after `app.use(express.json());`):

javascript

```javascript
const swaggerUi = require('swagger-ui-express');
const swaggerDocument = require('./openapi.json');

// Serve interactive Swagger UI documentation
app.use('/docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));
```

Use code with caution.

Python Lane (FastAPI Descriptions)

FastAPI creates your UI automatically at `http://localhost:8000/docs`. To clean it up and add a one-line summary to your endpoints, add descriptions directly inside your route decorators or docstrings.

Update your `main.py` file to include explicit summaries:

python

```python
from fastapi import FastAPI, HTTPException, status, Response
from pydantic import BaseModel, Field
from typing import Optional

app = FastAPI(title="Task API", version="1.0")

class TaskCreate(BaseModel):
    title: str = Field(..., min_length=1)

class TaskUpdate(BaseModel):
    title: Optional[str] = Field(None, min_length=1)
    done: Optional[bool] = None

tasks = [
    {"id": 1, "title": "Buy groceries", "done": False},
    {"id": 2, "title": "Clean the room", "done": True},
```

```
    {"id": 3, "title": "Study API design", "done": False}
]

@app.get("/", summary="Get API metadata")
def read_root():
    return {"name": "Task API", "version": "1.0", "endpoints": ["/tasks"]}

@app.get('/health', summary="Check API health status")
def read_health():
    return {"status": "ok"}

@app.get("/tasks", summary="List all tasks")
def read_tasks():
    return tasks

@app.get("/tasks/{id}", summary="Get a task by ID")
def read_task(id: int):
    task = next((t for t in tasks if t["id"] == id), None)
    if task is None:
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})
    return task

@app.post("/tasks", status_code=status.HTTP_201_CREATED, summary="Create a new task")
def create_task(task_input: TaskCreate):
    if not task_input.title.strip():
         raise HTTPException(status_code=400, detail={"error": "Title cannot be empty spac
    next_id = max([t["id"] for t in tasks], default=0) + 1
    new_task = {"id": next_id, "title": task_input.title, "done": False}
    tasks.append(new_task)
    return new_task

@app.put("/tasks/{id}", summary="Update an existing task")
def update_task(id: int, task_input: TaskUpdate):
    task = next((t for t in tasks if t["id"] == id), None)
    if task is None:
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})

    if task_input.title is None and task_input.done is None:
        raise HTTPException(status_code=400, detail={"error": "Missing fields to update"})

    if task_input.title is not None:
        if not task_input.title.strip():
            raise HTTPException(status_code=400, detail={"error": "Title cannot be empty s
        task["title"] = task_input.title
```

```
    if task_input.done is not None:
        task["done"] = task_input.done

    return task

@app.delete("/tasks/{id}", status_code=status.HTTP_204_NO_CONTENT, summary="Delete a task
def delete_task(id: int):
    global tasks
    task = next((t for t in tasks if t["id"] == id), None)
    if task is None:
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})

    tasks = [t for t in tasks if t["id"] != id]
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

Use code with caution.

🏁 Checkpoint Verification

1. **Open the browser:** Navigate to `http://localhost:3000/docs` (Express) or `http://localhost:8000/docs` (FastAPI).
2. **Review summaries:** Verify that all five endpoints are listed with clear human-readable names.
3. **Execute requests:** Click an endpoint, select **Try it out**, populate any required params/bodies, and hit **Execute** to see your live server responses right in the browser!

💾 Git Commit

Commit your progress in the terminal:

bash

```
git add .
git commit -m "Stage 5: Swagger UI"
```